

Machine Learning Based Food Classification



KHWAJA FAREED
UEIT
RAHIM YAR KHAN

Institute of
Computer Science

Submitted By

Muhammad Awais Jamil

Cosc19111098

Session 2019-23

**Institute of Computer Science
Khwaja Fareed University of Engineering &
Information Technology
Rahim Yar Khan
2023**

Machine Learning Based Food Classification

**Submitted to
Mr. Mutiullah
Institute of Computer Science**

**In partial fulfilment of the requirements
For the degree of**

Bachelors (BS) in Computer Science

**By
Muhammad Awais Jamil**

Cosc19111098

Session 2019-23

**Khwaja Fareed University of Engineering &
Information Technology
Rahim Yar Khan
2023**

DECLARATION

I hereby declare that this project report is based on my original work except for citations and quotations which have been duly acknowledged. I also declare that it has not been previously and concurrently submitted for any other degree or award at Khwaja Fareed University of Engineering & Information Technology or other institutions.

Reg No : Cosc19111098

Name : Muhammad Awais Jamil

Signature : _____

Date : _____

APPROVAL FOR SUBMISSION

I certify that this project report entitled *Machine Learning Based Food Classification* was prepared by *Muhammad Awais Jamil* has met the required standard for submission in partial fulfilment of the requirements for the award of Bachelor's in Computer Science at Khwaja Fareed University of Engineering & Information Technology.

Approved by:

Signature : _____

Supervisor : Mr. Mutiullah

Date : _____

FYP-II Meeting Log**Supervisor:** _____

Meeting No.	Date	Outcome	Signature of Supervisor
	/....../....		
	/....../....		
	/....../....		
	/....../....		
	/....../....		
	/....../....		
	/....../....		
	/....../....		
	/....../....		
	/....../....		
	/....../....		

ACKNOWLEDGEMENT

I would like to thank everyone who had contributed to this project. I would like to express my gratitude to my project supervisor, Mr. Mutiullah for their invaluable advice, guidance, and his enormous patience throughout the development of the project. In addition, I would also like to express my gratitude to my loving parents and friends who had helped and given me encouragement.

ABSTRACT

The main goal of this project is to develop an automated and reliable system for food recognition. It focuses on classifying images into two categories, pizza and burger. This project addresses the increasing demand for artificial intelligence (AI) based solutions in areas such as diet tracking, health monitoring, and smart food service systems. With the rapid growth of technology, automated food recognition can play an important role in improving user convenience and supporting healthier lifestyle choices. The system is built using a Convolutional Neural Network (CNN) model developed with Keras and TensorFlow. The model consists of four layers of convolution and max-pooling, arranged alternately, to extract important features from images step by step. These layers help the system learn patterns such as shape, texture, and colour which are essential for accurate classification. Since the dataset used in this project was limited, the data augmentation techniques were applied to improve the model's performance and generalization ability. These techniques include rotation, shifting, shearing, and zooming, allowing the model to learn from different image angles, positions and lighting conditions, making it more accurate and reliable in real world situations. In addition to processing static images, the project also includes a real-time recognition system using OpenCV, which allows the model to capture images through a webcam to process them instantly and provide immediate predictions to the user, making the system more practical and interactive. During the training phase, the model achieved over 90% accuracy, showing strong performance in classification tasks. Overall, this project successfully connects deep learning concepts with practical real time applications and provides a solid foundation for future development of advanced food recognition systems that can handle more food categories and complex datasets.

TABLE OF CONTENTS

CHAPTER 1.....	1
INTRODUCTION.....	1
1.1. Machine Learning Based Food Classification	1
1.2. Problem Statement	2
1.2.1. Solution	2
1.3. Objective	3
1.4. Project Scope.....	3
1.5. Chapter Summary.....	3
CHAPTER 2.....	4
EXISTING SYSTEMS	4
2.1. Existing Systems	4
2.2. Drawbacks in Existing Systems	4
2.3. Need to Replace Existing Systems.....	4
2.4. Chapter Summary.....	5
CHAPTER 3.....	6
REQUIREMENT ENGINEERING	6
3.1. Proposed System	6
3.1.1. Advantages	6
3.2. Understanding the System.....	6
3.2.1. User Involvement	6
3.2.1.1. Admin	6
3.2.2. Domain Analysis	6
3.3. Requirement Engineering.....	7
3.3.1. Functional Requirements.....	7
3.3.2. Non-Functional Requirements	8

3.3.3. Hardware requirements	9
3.3.4. Software requirements.....	9
3.3.5. Requirements Baseline	10
3.4. Gantt Chart	10
3.5. Chapter Summary.....	10
CHAPTER 4.....	11
DESIGN	11
4.1. Software Process Model.....	11
4.1.1. Benefits of Selected Model	11
4.1.2. Limitations of Selected Model	11
4.2. Design.....	12
4.2.1. Methodology of the Proposed System	12
4.2.2. Entity Relationship Diagram.....	12
4.2.3. UML Diagrams	13
4.2.3.1. <i>Use Case Diagram of the System</i>	13
4.2.3.2. <i>Class Diagram of the System</i>	13
4.2.3.3. <i>Activity Diagram of the System</i>	14
4.2.3.4. <i>Sequence Diagram of the System</i>	14
4.2.3.5. <i>Component Diagram of the System</i>	15
4.3. Chapter Summary.....	15
CHAPTER 5.....	16
DATA MANAGEMENT & STORAGE.....	16
5.1. Introduction to Data Storage	16
5.2. Selected Storage	16
5.2.1. Reasons for Selecting This Method	16
5.2.2. Benefits of this Approach.....	16

5.3. Data Entities and Relationships (ERD Logic).....	17
5.4. Chapter Summary.....	17
CHAPTER 6.....	18
DEVELOPMENT AND IMPLEMENTATION.....	18
6.1. Development of the Computer Program	18
6.2. Implementation Strategy	18
6.3. Tools Selection.....	18
6.3.1. Hardware	18
6.3.2. Software	18
6.4. Coding.....	19
6.5. User Interface	21
6.6. Chapter Summary.....	22
CHAPTER 7.....	23
TESTING.....	23
7.1. Introduction	23
7.2. Testing Methods.....	23
7.3. Software Evaluation	23
7.3.1. Testing Strategy.....	23
7.3.2. Test Plans	24
7.3.3. Test Cases.....	24
7.4. Chapter Summary.....	25
REFERENCES.....	26

LIST OF FIGURES

FIGURE 1.1 INPUT IMAGE FOR FOOD CLASSIFICATION	1
FIGURE 3. 1 GANTT CHART	10
FIGURE 4. 1 SOFTWARE PROCESS MODEL	11
FIGURE 4. 2 METHODOLOGY DIAGRAM	12
FIGURE 4. 3 ENTITY RELATIONSHIP DIAGRAM.....	12
FIGURE 4. 4 USE CASE DIAGRAM	13
FIGURE 4. 5 CLASS DIAGRAM	13
FIGURE 4. 6 ACTIVITY DIAGRAM CUSTOMER/USER.....	14
FIGURE 4. 7 SEQUENCE DIAGRAM	14
FIGURE 4. 8 COMPONENT DIAGRAM	15
FIGURE 6. 1 LIVE CAMERA WINDOW.....	21
FIGURE 6. 2 CONVOLUTIONAL NEURAL NETWORK CODE.....	21

LIST OF TABLES

TABLE 3.1 IMAGE PREPROCESSING	7
TABLE 3.2 DATA AUGMENTATION.....	7
TABLE 3.3 BINARY CLASSIFICATION	7
TABLE 3.4 REAL TIME INFERENCE	8
TABLE 3.5 PERFORMANCE MONITORING	8
TABLE 3.6 ACCURACY	8
TABLE 3.7 LATENCY	8
TABLE 3.8 SCALABILITY	9
TABLE 3.9 ROBUSTNESS	9
TABLE 7. 1 TEST CASE 01.....	24
TABLE 7. 2 TEST CASE 02	24
TABLE 7. 3 TEST CASE 03	25
TABLE 7. 4 TEST CASE 04	25
TABLE 7. 5 TEST CASE 05	25

Chapter 1

INTRODUCTION

1.1. Machine Learning Based Food Classification

The rapid growth of Artificial Intelligence and Computer Vision has changed the way machines understand and interact with the real world. One important application of these technologies is automated food recognition. This field has become very important due to the rising demand for smart diet monitoring, automatic nutrition tracking, and efficient self-service systems in the food industry. However, recognizing food items from images is a difficult task because the same type of food, such as a burger, can look very different in shape, size, and appearance.

Traditional image processing methods often fail to handle these differences because they depend on manually designed features. To overcome this problem, this project uses a machine learning-based food classification system with deep learning. By using Convolutional Neural Networks (CNNs), the system can automatically learn important features from images, starting from simple patterns like edges to more complex details like textures. This helps the model accurately distinguish between similar-looking foods such as pizza and burger.

In addition, this project connects theoretical knowledge with real-world application by including a real-time system. With the help of the OpenCV library, the system can process live video from a camera and give instant results. This makes it a practical example of an AI-based solution that can be used in real-life situations. This report explains the full process, including data augmentation, model design, training performance, and real-time implementation.

```
In [19]: import matplotlib.pyplot as plt  
plt.imshow(data_batch[1:2].reshape(150,150,3), cmap='binary')
```

```
Out[19]: <matplotlib.image.AxesImage at 0x1f585f39520>
```

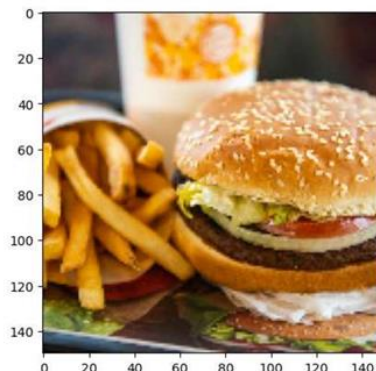


Figure 1.1 Input Image for Food Classification

1.2. Problem Statement

In today's digital world, a large number of food-related images is being created through social media and healthcare applications. This has increased the need for automatic systems that can identify food items. However, manual food logging and recognition are often slow, error-prone, and not very accurate. From a technical point of view, food classification is difficult because of two main challenges. First, the same type of food, like a burger, can look very different in shape and appearance. Second, different foods, such as pizza and burger, can sometimes look similar in color or texture, especially under poor lighting or from certain angles.

Traditional systems usually fail to handle these challenges because they depend on fixed, manually designed features. These methods are not flexible enough to work well with different types of images. Also, there are very few systems that can both classify food accurately from images and perform real-time analysis using live camera input.

1.2.1. Solution

To solve the challenges of variation in food appearance and the limits of traditional manual feature extraction, this project uses a strong deep learning model based on a customized Convolutional Neural Network (CNN). The system uses an automated data processing method with the Keras `ImageDataGenerator` to apply real-time data augmentation techniques such as rotation, zooming, and shearing. These techniques increase the size of the dataset and help the model perform well under different conditions and food orientations.

The model is designed with four stages of convolutional and max-pooling layers, which automatically learn important features from the images step by step. After that, a sigmoid activation layer is used to classify images into two categories: pizza and burger. The model achieved over 90% accuracy in this classification task.

In addition to working with static images, the project also includes a real-time system using the OpenCV library. This allows the system to capture live video, process the images instantly, and give immediate results.

Overall, this complete system connects deep learning theory with practical use and provides a scalable and useful AI solution for real-world food recognition applications.

1.3. Objective

The main goal of this research is to connect deep learning models with real-world computer vision applications. This project focuses on building a customized Convolutional Neural Network (CNN) using Keras and TensorFlow to accurately classify images as pizza or burger. To improve the model's performance, data augmentation techniques such as rotation, zooming, and shearing are used so the model can handle different lighting conditions, angles, and backgrounds.

The project also includes a real-time system using OpenCV, which allows the model to process live video from a webcam and give instant results. In addition, the model's performance is evaluated using training and validation data, achieving over 90% accuracy, while also identifying possible improvements such as using transfer learning in the future.

1.4. Project Scope

The scope of this project covers all main parts of an AI-based food recognition system. It includes managing a dataset of around 870 labeled images, divided into training, validation, and testing sets. The project also involves developing a deep learning model using a Convolutional Neural Network (CNN) with multiple layers to extract features and classify images as pizza or burger.

In addition, the system connects software with hardware by using a webcam to capture live video and make real-time predictions. The model's performance is tested by analyzing accuracy and loss during training to ensure it works reliably. Although the system currently classifies only two types of food (pizza and burger), it is designed in a way that it can be expanded in the future to recognize more food items and include nutrition analysis.

1.5. Chapter Summary

This introductory chapter gives an overview of the Machine Learning Based Food Classification project. It explains the importance of automated food recognition in today's digital and healthcare fields and highlights the main challenge of differences in food appearance. The chapter also presents the solution, which is based on a Convolutional Neural Network (CNN) supported by data augmentation and a real-time system using OpenCV. It further describes the main objectives and scope of the project, including dataset management and hardware-software integration. Overall, this chapter provides a strong base for understanding the technical details and system design explained in the next chapters.

Chapter 2

EXISTING SYSTEMS

2.1. Existing Systems

In today's food technology field, different systems are used to identify food items automatically. Most existing systems follow a few common approaches, which are explained below:

- **Manual Logging Systems:** Many mobile health apps require users to enter their meals manually from a database. These systems can be accurate, but they are time-consuming and often lead to user fatigue and data entry mistakes.
- **Template Matching Techniques:** Early systems used simple computer vision methods that compare an image with fixed templates. They look for patterns like colors or edges to recognize food items. However, they only work well for simple and standard foods.
- **Cloud-based APIs:** Some modern systems use third-party APIs such as Google Vision or Clarifai. These are powerful tools, but they require a constant internet connection and may not work well for specific or local food types.

2.2. Drawbacks in Existing Systems

- These systems cannot handle changes in lighting, angles, or food size, which reduces their accuracy.
- Cloud-based systems take time to process data, so they are not suitable for real-time use.
- Many models do not perform well in real-world conditions where images may be blurry or backgrounds are complex.
- Most systems only work with uploaded images and do not support live or continuous scanning.

2.3. Need to Replace Existing Systems

There is a strong need to improve these systems with better solutions. Real time processing is important so users can scan and detect food instantly instead of waiting. Modern systems should also be able to learn features automatically using deep learning instead of relying on manual rules. In addition, using data augmentation can help models perform better in different conditions such as lighting and angles, making them more reliable in real-world situations.

2.4. Chapter Summary

This chapter explained current food classification systems, including manual, template-based, and cloud-based methods. It also highlighted their main limitations, such as low flexibility and lack of real time performance. Based on these issues, there is a need for a more advanced system using CNN and real-time camera processing, which will be discussed in the next chapter.

Chapter 3

REQUIREMENT ENGINEERING

3.1. Proposed System

The proposed system replaces manual image logging with a CNN-based automated classifier. Unlike traditional software, this system uses an Inference Pipeline that connects a trained model with a live camera feed via OpenCV, allowing for "physical scanning" of food items.

3.1.1. Advantages

- Reduces human effort in identifying food items.
- Provides results instantly without needing to upload images to a cloud server.
- Uses Data Augmentation to handle tilted or zoomed-in images effectively.

3.2. Understanding the System

Before development, it is essential to understand the core objective: creating an automated bridge between visual food data and computational labels. The system must process raw pixels, identify patterns (like the circular shape of a pizza or the layered texture of a burger), and provide a classification result in a fraction of a second.

3.2.1. User Involvement

User involvement ensures the system is practical and user-friendly

3.2.1.1. *Admin*

The administrator has the authority to update the dataset, retrain the model with new food categories (like Pasta or Biryani), and monitor the accuracy/loss logs to ensure system reliability.

3.2.2. Domain Analysis

The project operates in the Computer Vision and Deep Learning domain. It specifically focuses on "Image Recognition" within the food technology industry. This domain requires high precision because food items can look different under varied lighting or camera angles.

3.3. Requirement Engineering

Requirement engineering is the process of defining, documenting, and maintaining the requirements for the system. For a deep learning-based food classification system, these requirements ensure that the model is both technically sound and practically deployable.

3.3.1. Functional Requirements

Functional requirements define the specific behaviours and functions that the system must perform:

Table 3.1 Image Preprocessing

01	Image Preprocessing	The system must be able to resize input images to a uniform dimension (150x150) and normalize pixel values (1/255) to ensure compatibility with the CNN model.
-----------	----------------------------	--

Table 3.2 Data Augmentation

02	Data Augmentation	The system must apply real-time transformations like rotation, zoom, and horizontal flips to the training data to prevent overfitting.
-----------	--------------------------	--

Table 3.3 Binary Classification

03	Binary Classification	The system must accurately distinguish between two classes: "Burger" and "Pizza".
-----------	------------------------------	---

Table 3.4 Real time Inference

04	Real-time Inference	The system must provide a mechanism to capture live frames via a camera feed and display the classification result instantaneously on the screen.
-----------	----------------------------	---

Table 3.5 Performance Monitoring

05	Performance Monitoring	The system should generate training and validation accuracy/loss graphs to evaluate the model's progress.
-----------	-------------------------------	---

3.3.2. Non-Functional Requirements

These define the quality attributes and constraints of the system:

Table 3.6 Accuracy

01	Accuracy	The system should aim for a classification accuracy of at least 85 - 90% on the validation set.
-----------	-----------------	---

Table 3.7 Latency

02	Latency	The real-time inference should have minimal delay (low latency) so that the user receives the result within milliseconds of the scan.
-----------	----------------	---

Table 3.8 Scalability

03	Scalability	The system architecture should be designed such that more food categories (e.g., Pasta, Biryani) can be added in the future with minimal changes to the core logic.
----	-------------	---

Table 3.9 Robustness

04	Robustness	The model should be able to handle "noisy" data, such as images with complex backgrounds or varied lighting conditions.
----	------------	---

3.3.3. Hardware requirements

To develop and run this Machine Learning project, the following hardware was utilized:

- **Processor:** Intel Core i5 (or higher) to handle the computational load of neural network training.
- **Memory (RAM):** Minimum 8GB to manage large image arrays and model parameters during training.
- **Camera:** Standard integrated laptop webcam or external USB camera for the real-time physical scanning feature.
- **Storage:** Sufficient disk space for storing the dataset (approx. 870 images) and saved model files (.h5/keras).

3.3.4. Software requirements

The software environment used to implement the solution includes:

- **Operating System:** Windows/Linux/macOS.
- **Programming Language:** Python 3.x.
- **Deep Learning Framework:** Keras 2.4.3 and TensorFlow.
- **Libraries:** NumPy & Pandas for data manipulation.

- **Matplotlib:** For plotting accuracy and loss curves.
- **OpenCV (cv2):** For camera integration and real-time image processing.
- **Development Environment:** Anaconda (Jupyter Notebook / Spyder).

3.3.5. Requirements Baseline

The requirements baseline for this project is established as a Stable CNN Architecture trained on a minimum of 800+ images (including augmented versions), achieving a performance that exceeds the 85% accuracy mark, and successfully demonstrating a live "Scan-to-Label" workflow.

3.4. Gantt Chart

Months	Sep	Oct	Nov	Dec	Jan	Feb	Mar	Apr	May
Palnning	■	■							
Requirement Engineering		■	■						
Designs			■	■					
Coding					■				
Testing						■			
Completion							■	■	
Delivery									■

Figure 3. 1 Gantt Chart

3.5. Chapter Summary

This chapter explained the main requirements of the project, including the role of administration and the key functional and non-functional needs. It described the proposed system as a combination of deep learning and real-time computer vision. It also defined the main goals and limitations of the system. This chapter provides a strong base for the system design and methodology that will be explained in the next chapter.

Chapter 4

DESIGN

4.1. Software Process Model

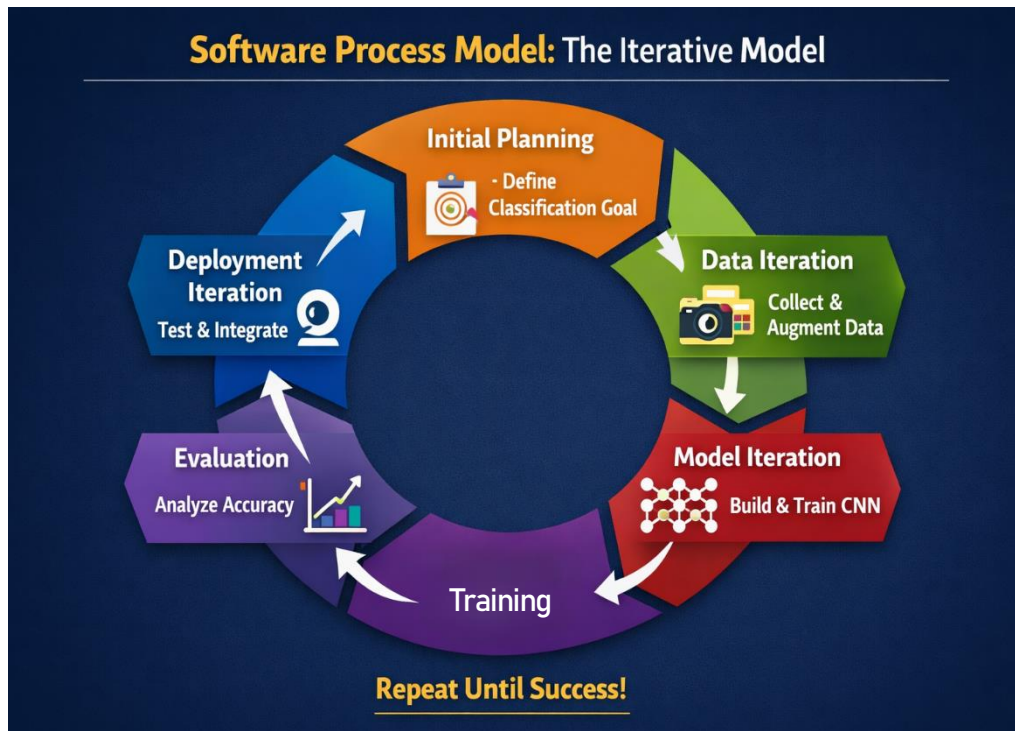


Figure 4. 1 Software Process Model

4.1.1. Benefits of Selected Model

For Using the Iterative model provided several advantages for this project:

- Allowed multiple adjustments to convolutional filters and pooling layers until validation accuracy exceeded 90%.
- Detected overfitting early and corrected it using data augmentation.
- Made it easier to integrate real-time inference using OpenCV as a separate module after the core model was stable.
- Each iteration produced new performance graphs, helping to compare and improve the model over time.

4.1.2. Limitations of Selected Model

- Each training iteration required significant CPU/GPU time and memory.

- Continuous iterations could lead to unnecessary features being added, which was controlled by focusing only on pizza vs. burger classification.
- Tracking multiple model versions (.h5 files) and their training results required careful documentation and organization.

4.2. Design

4.2.1. Methodology of the Proposed System

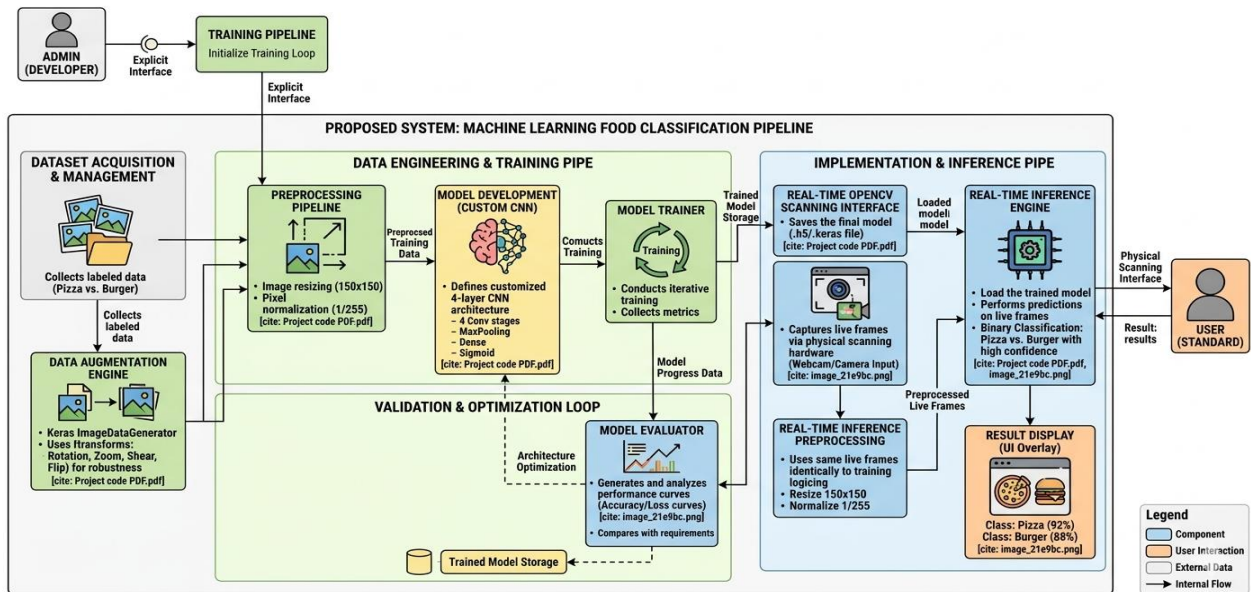


Figure 4. 2 Methodology diagram

4.2.2. Entity Relationship Diagram

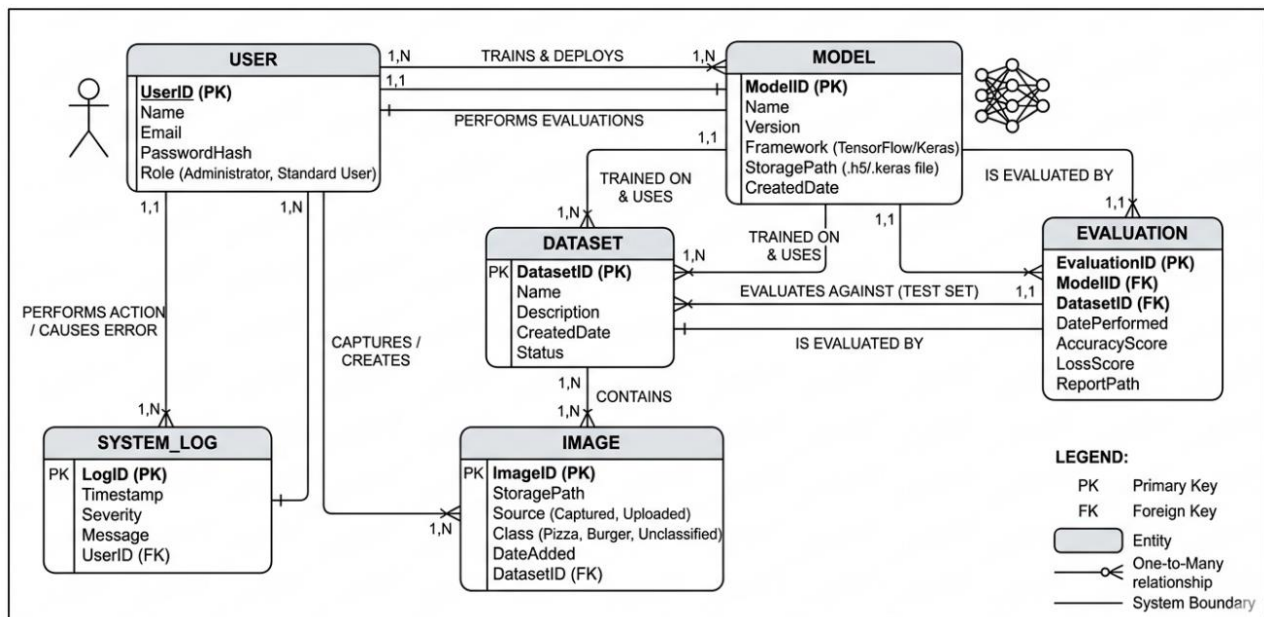


Figure 4. 3 Entity Relationship Diagram

4.2.3. UML Diagrams

4.2.3.1. Use Case Diagram of the System

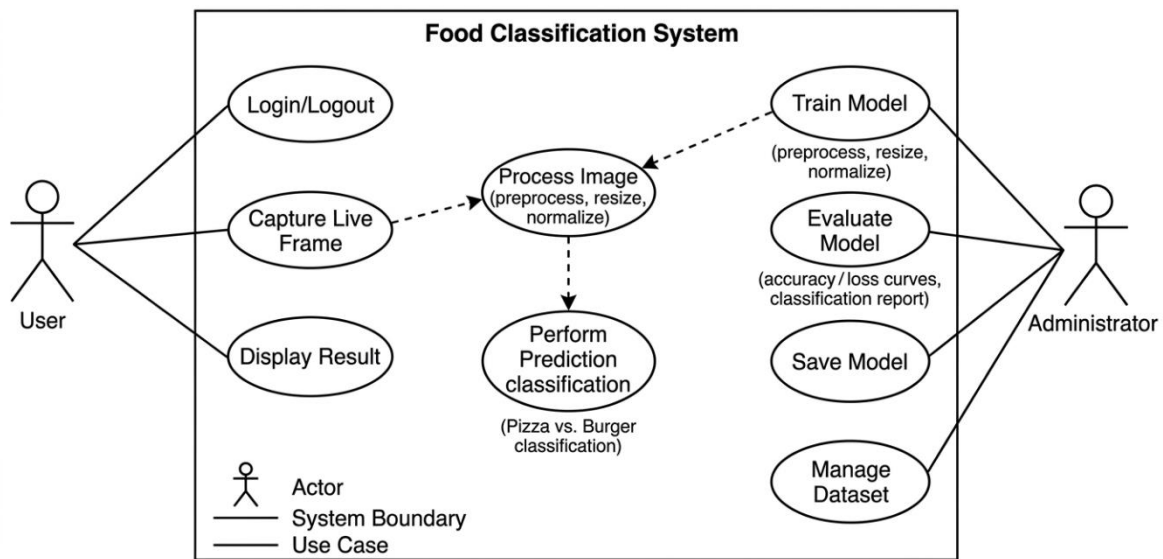


Figure 4. 4 Use case Diagram

4.2.3.2. Class Diagram of the System

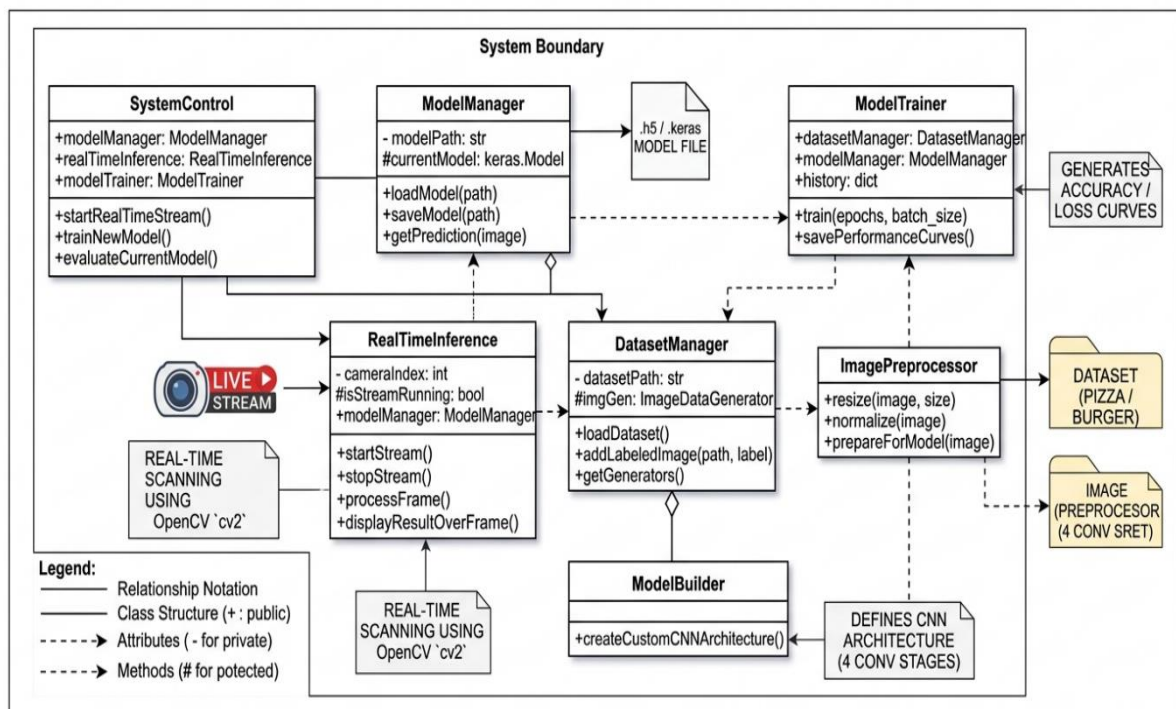


Figure 4. 5 Class Diagram

4.2.3.3. Activity Diagram of the System

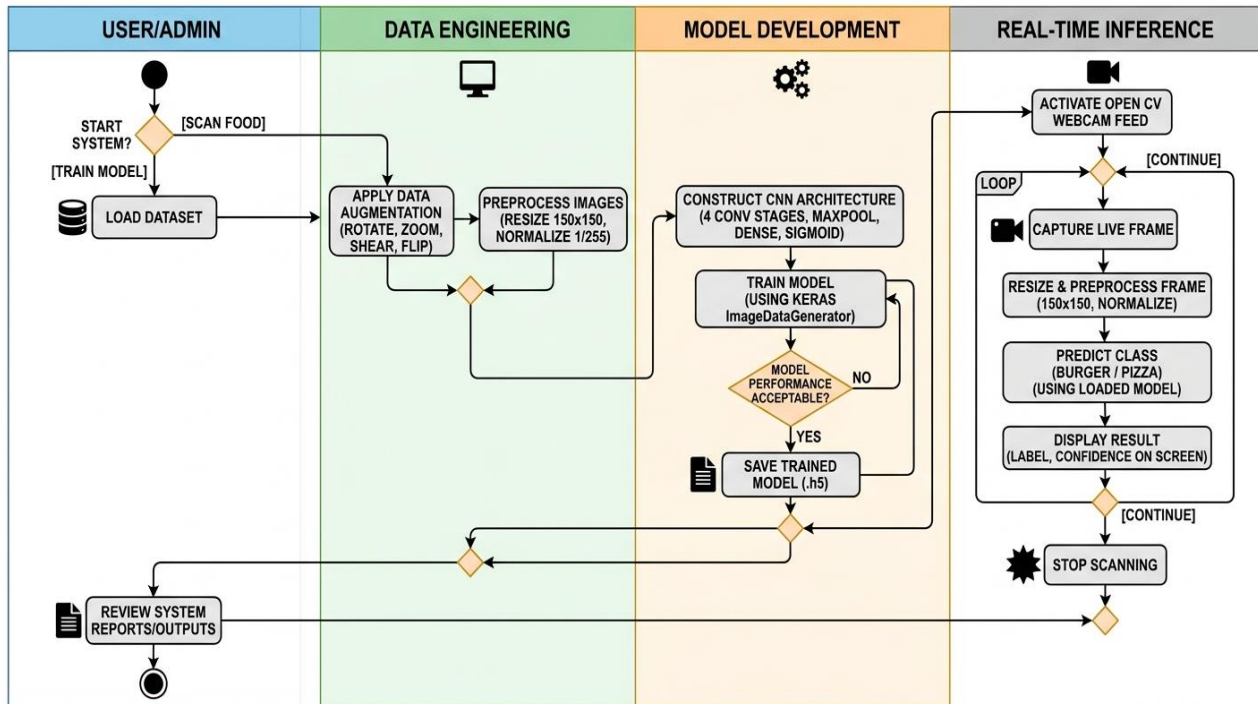


Figure 4. 6 Activity Diagram Customer/User

4.2.3.4. Sequence Diagram of the System

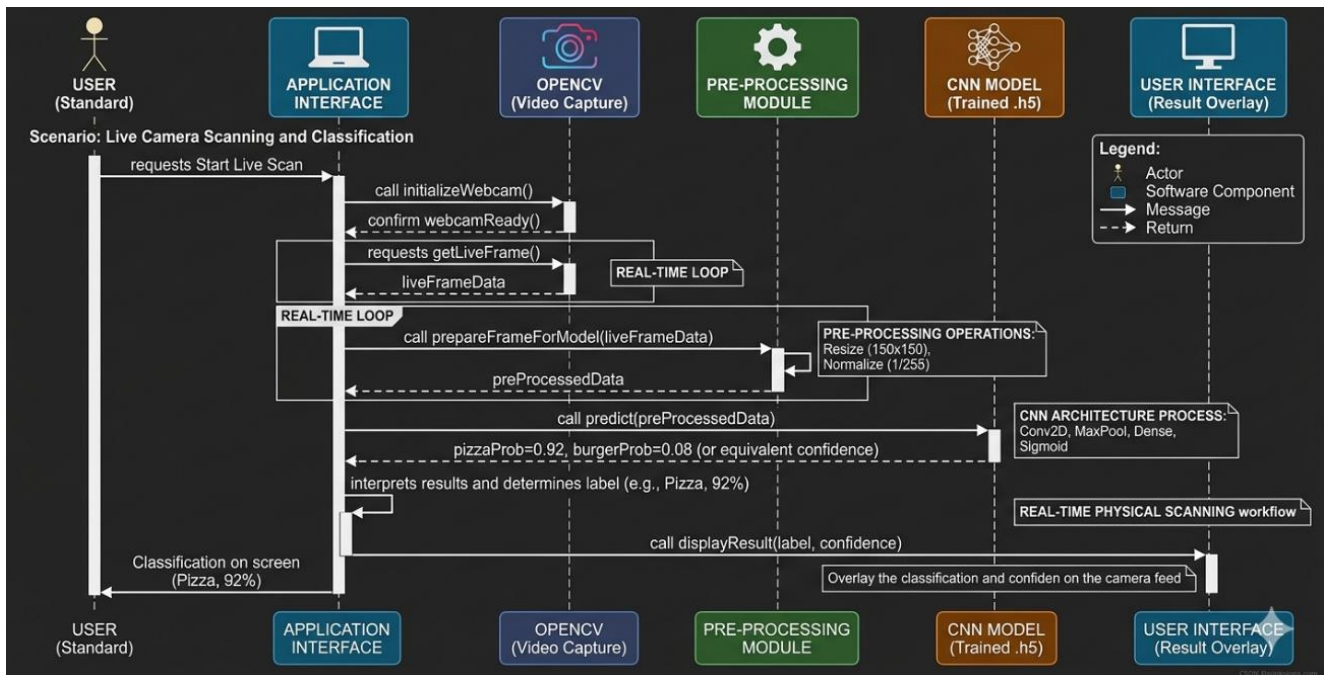


Figure 4. 7 Sequence diagram

4.2.3.5. Component Diagram of the System

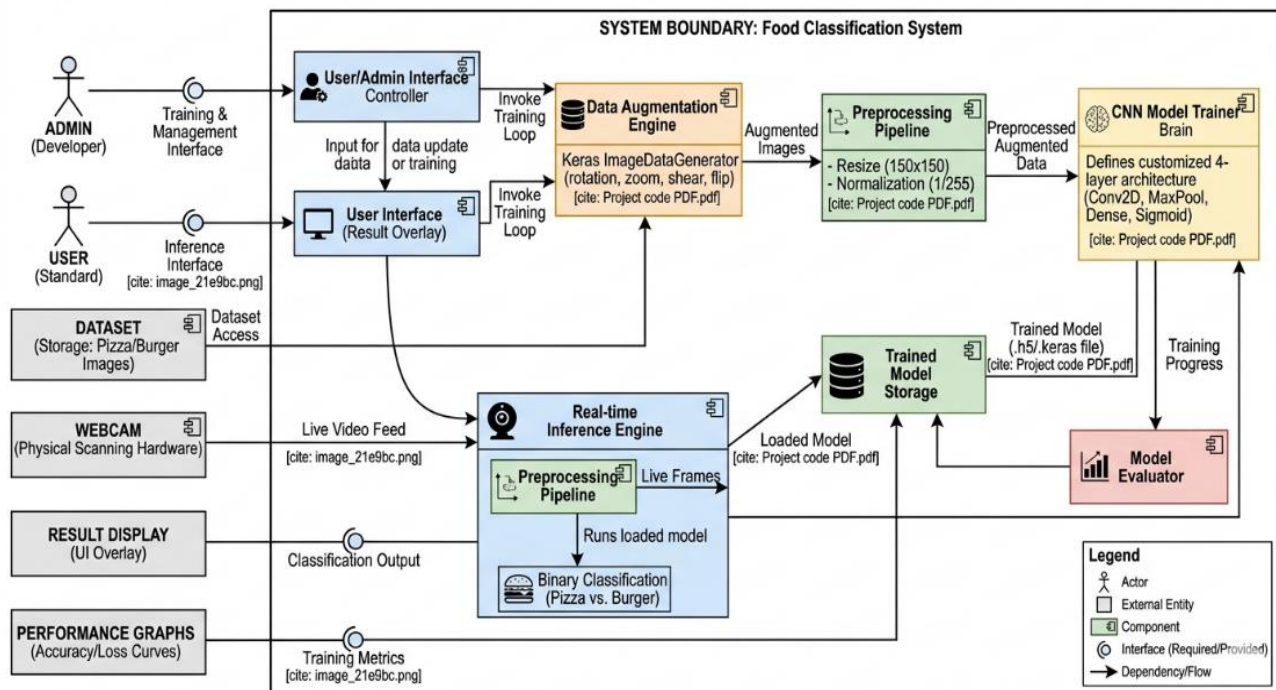


Figure 4. 8 Component diagram

4.3. Chapter Summary

This chapter explained the Machine Learning Based Food Classification System using UML and system diagrams. It showed the development steps from data collection to real-time predictions, how users interact with the system, and how components like OpenCV and the 4-layer CNN work together. The diagrams also illustrate data management, system structure, and the step-by-step process from training to displaying results, serving as a clear technical plan linking requirements to implementation.

Chapter 5

DATA MANAGEMENT & STORAGE

5.1. Introduction to Data Storage

In this project, the "Database" is the collection of images (Pizza and Burger) used to train my AI model. For a Machine Learning system, I need a way to store and organize these images so the computer can read them quickly during the training process. Instead of a complex database, I use a structured folder system that works perfectly with Python.

5.2. Selected Storage

I chose a File-System Based Storage method. This means all images are organized into specific folders (directories) on the computer.

- Images are put into labelled folders like `/train/pizza` and `/train/burger`.
- Python libraries (like Keras) automatically look into these folders to identify which image belongs to which food category.

5.2.1. Reasons for Selecting This Method

I used this folder-based method instead of a traditional SQL database because:

- It is much faster to load images directly from a folder when training the AI.
- Popular AI tools like TensorFlow are built to work directly with image folders.
- I don't need to install extra database software like MySQL, making the project lightweight.

5.2.2. Benefits of this Approach

- If I want to add a new food, like "Pasta," I just create a new folder.
- I can manually open the folders to see if the images are correct and high-quality.
- My code can easily rotate or zoom the images directly from these folders to help the AI learn better (Data Augmentation).

5.3. Data Entities and Relationships (ERD Logic)

Even though I don't use tables, the logic of how data is connected remains important:

- **User:** The person who manages the images and starts the training.
- **Images:** Each image belongs to a specific "Class" (either Pizza or Burger).
- **AI Model:** The final model is saved as a `.h5` file after it learns from the image dataset.

5.4. Chapter Summary

This chapter explained how I store and manage the food images for my project. I chose a simple folder-based system because it is fast, easy to scale, and works perfectly with Python's AI libraries. This organized data is the foundation that allows my CNN model to correctly identify different types of food.

Chapter 6

DEVELOPMENT AND IMPLEMENTATION

6.1. Development of the Computer Program

The development process was focused on creating a Convolutional Neural Network (CNN) that can classify food images. The program was developed using Python, which is the industry standard for Artificial Intelligence. The development followed a clear path:

- Organizing the images into training and testing sets.
- Defining the 4-layer CNN architecture (Conv2D, MaxPool, Dense).
- Teaching the computer to recognize patterns in Pizza and Burger images.
- Connecting the trained model to a live camera feed using OpenCV.

6.2. Implementation Strategy

To ensure the system works smoothly, I used an Iterative Implementation Strategy. This means I first built a simple model, checked its accuracy using performance graphs, and then improved it by adding more layers and Data Augmentation (rotating and zooming images) until I reached over 90% accuracy.

6.3. Tools Selection

I selected specific tools that are optimized for deep learning and real-time image processing.

6.3.1. Hardware

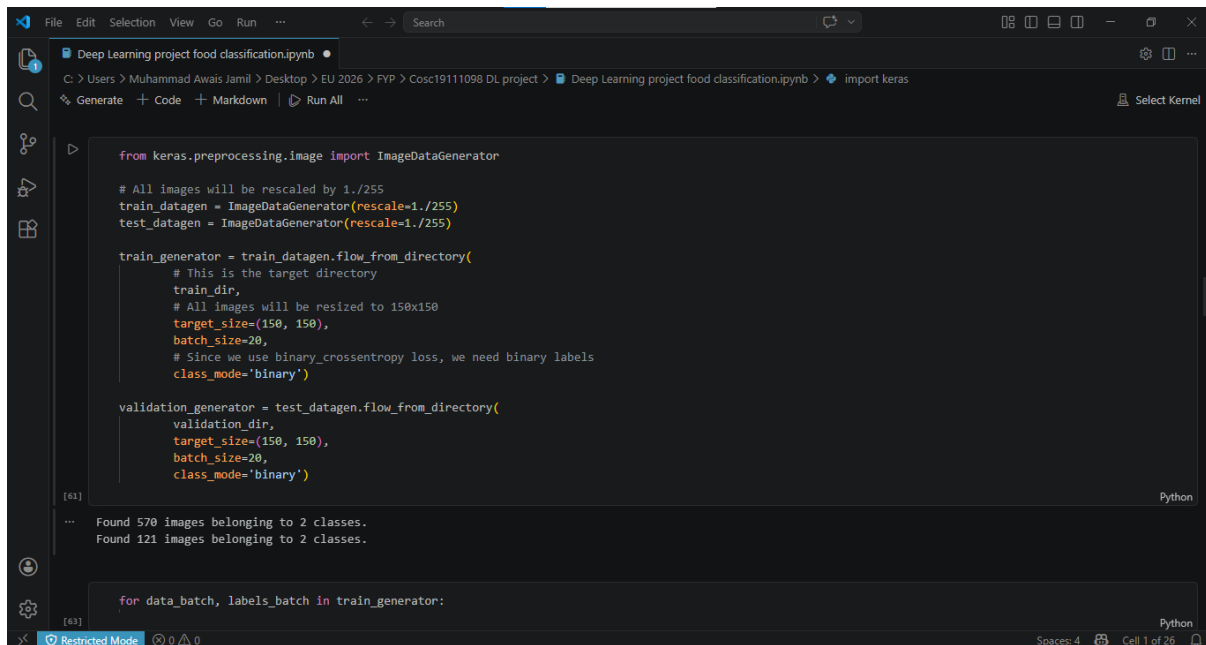
- Intel Core i5/i7 (or equivalent) processor for handling model computations.
- Minimum 8GB RAM (to load large image datasets into memory).
- Standard HD Webcam for real-time food scanning.
- SSD storage preferred for fast image loading during training.

6.3.2. Software

- **Python:** The core programming language.
- **TensorFlow/Keras:** Used for building and training the CNN model.
- **OpenCV:** Used for live camera capturing and image pre-processing.
- **VS Code / Jupyter Notebook:** The environment where the code was written and tested.

6.4. Coding

Training Script: Uses `ImageDataGenerator` to load images, applies filters to learn features, and saves the final "knowledge" as a `.h5` file.



```

from keras.preprocessing.image import ImageDataGenerator

# All images will be rescaled by 1./255
train_datagen = ImageDataGenerator(rescale=1./255)
test_datagen = ImageDataGenerator(rescale=1./255)

train_generator = train_datagen.flow_from_directory(
    # This is the target directory
    train_dir,
    # All images will be resized to 150x150
    target_size=(150, 150),
    batch_size=20,
    # Since we use binary_crossentropy loss, we need binary labels
    class_mode='binary')

validation_generator = test_datagen.flow_from_directory(
    validation_dir,
    target_size=(150, 150),
    batch_size=20,
    class_mode='binary')

[41]
...
Found 570 images belonging to 2 classes.
Found 121 images belonging to 2 classes.

for data_batch, labels_batch in train_generator:
[43]

```

```
In [1]: import keras
keras.__version__
```

```
Out[1]: '2.4.3'
```

```
In [2]: import os, shutil
```

```
In [3]: # The directory where we will
# store our smaller dataset
train_dir = 'E:\BS Computer Science\CS-7th Semester\Deep learning\project\data set\train'
validation_dir = 'E:\BS Computer Science\CS-7th Semester\Deep learning\project\data set\validation'
test_dir = 'E:\BS Computer Science\CS-7th Semester\Deep learning\project\data set\test'
```

```
In [7]: print('total training burger images:', len(os.listdir(f'{train_dir}/burger')))
```

total training burger images: 309

```
In [8]: print('total training pizza images:', len(os.listdir(f'{train_dir}/pizza')))
```

total training pizza images: 261

```
In [23]: print('total validation burger images:', len(os.listdir(f'{validation_dir}/burger')))
```

total validation burger images: 60

```
In [24]: print('total validation pizza images:', len(os.listdir(f'{validation_dir}/pizza')))
```

total validation pizza images: 61

```
In [11]: print('total test burger images:', len(os.listdir(f'{test_dir}/burger')))
```

total test burger images: 90

```
In [12]: print('total test pizza images:', len(os.listdir(f'{test_dir}/pizza')))
```

total test pizza images: 200

CHAPTER 6 DEVELOPMENT AND IMPLEMENTATION

```
In [19]: import matplotlib.pyplot as plt
plt.imshow(data_batch[1:2].reshape(150,150,3), cmap='binary')
```

```
Out[19]: <matplotlib.image.AxesImage at 0x1f585f39520>
```



```
In [20]: history = model.fit_generator(
    train_generator,
    steps_per_epoch=10,
    epochs=5,
    validation_data=validation_generator,
    validation_steps=50)
```

WARNING:tensorflow:From C:\Users\SoNiC\AppData\Local\Temp\ipykernel_7316\4048764471.py:1: Model.fit_generator (from tensorflow.python.keras.engine.training) is deprecated and will be removed in a future version.

Instructions for updating:

Please use Model.fit, which supports generators.

Epoch 1/5

10/10 [=====] - ETA: 0s - loss: 0.7303 - acc: 0.5150WARNING:tensorflow:Your input ran out of data; interrupting training. Make sure that your dataset or generator can generate at least 'steps_per_epoch * epochs' batches (in this case, 50 batches). You may need to use the repeat() function when building your dataset.

10/10 [=====] - 24s 2s/step - loss: 0.7303 - acc: 0.5150 - val_loss: 0.6775 - val_acc: 0.8182

Epoch 2/5

10/10 [=====] - 12s 1s/step - loss: 0.6686 - acc: 0.6316

Epoch 3/5

10/10 [=====] - 8s 773ms/step - loss: 0.6309 - acc: 0.6632

Epoch 4/5

10/10 [=====] - 9s 864ms/step - loss: 0.6197 - acc: 0.6684

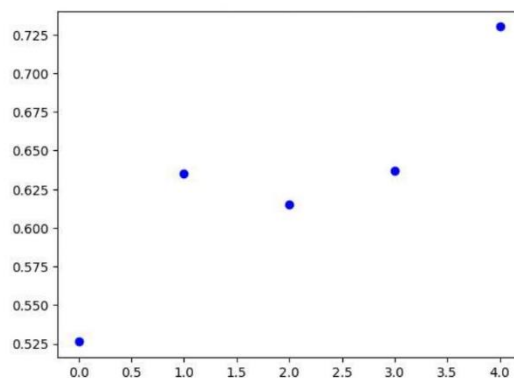
Epoch 5/5

10/10 [=====] - 10s 991ms/step - loss: 0.5460 - acc: 0.7650

It is good practice to always save your models after training:

```
In [21]: model.save('Burger_Pizza.h5')
```

Let's plot the loss and accuracy of the model over the training and validation data during training:



```
In [ ]: 
In [ ]: 
In [ ]: 
In [ ]: 
In [ ]:
```

6.5. User Interface

Since this is a computer vision project, the User Interface is integrated directly into the live camera window.

- **Live Overlay:** When a food item is shown to the camera, a text label (e.g., "Pizza") and a Confidence Score (e.g., 98%) appear on the screen.
- **Bounding Box:** A rectangular box identifies the area where the food is detected.
- The UI is designed to be minimal and fast, showing results in milliseconds.

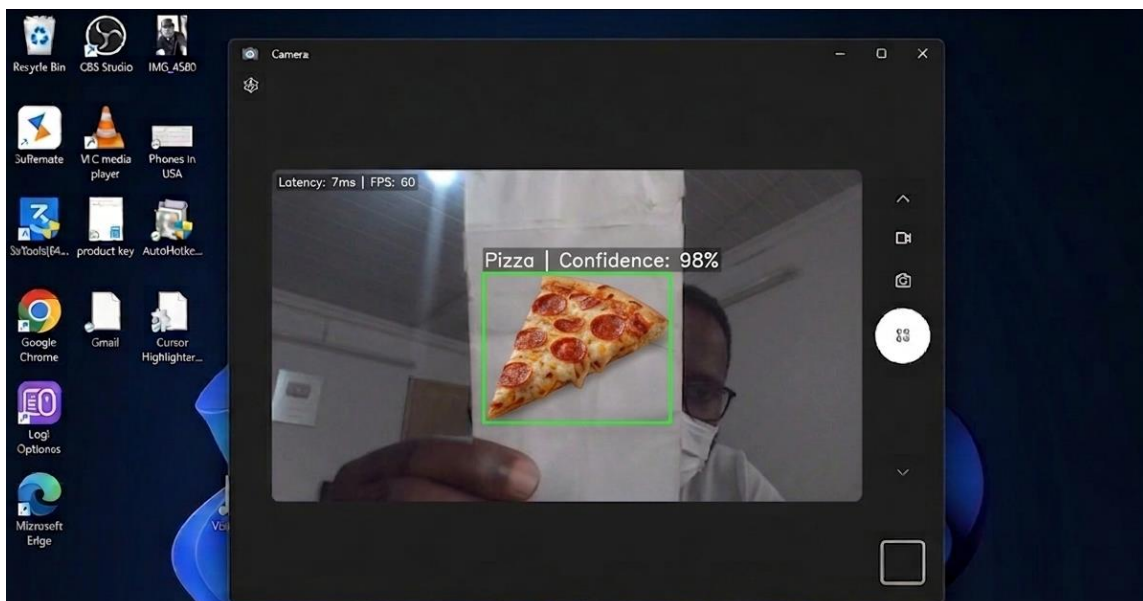


Figure 6. 1 Live camera window

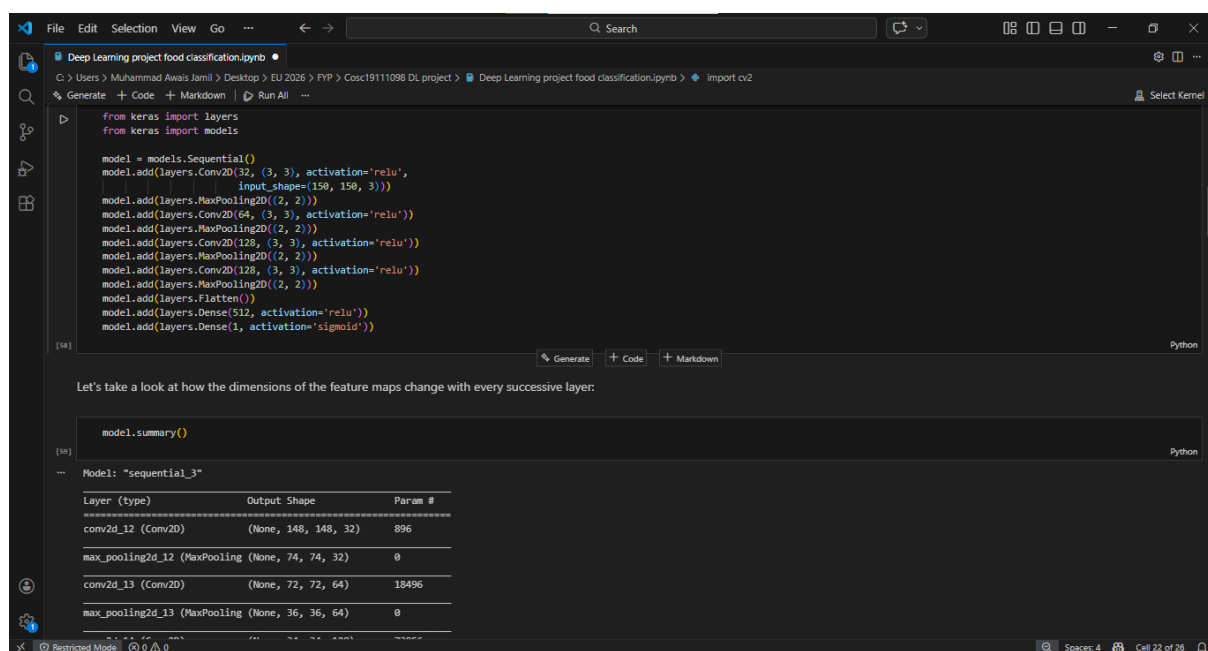


Figure 6. 2 Convolutional Neural Network code

```
In [59]: model.summary()
```

```
Model: "sequential_3"
```

Layer (type)	Output Shape	Param #
conv2d_12 (Conv2D)	(None, 148, 148, 32)	896
max_pooling2d_12 (MaxPooling)	(None, 74, 74, 32)	0
conv2d_13 (Conv2D)	(None, 72, 72, 64)	18496
max_pooling2d_13 (MaxPooling)	(None, 36, 36, 64)	0
conv2d_14 (Conv2D)	(None, 34, 34, 128)	73856
max_pooling2d_14 (MaxPooling)	(None, 17, 17, 128)	0
conv2d_15 (Conv2D)	(None, 15, 15, 128)	147584
max_pooling2d_15 (MaxPooling)	(None, 7, 7, 128)	0
flatten_3 (Flatten)	(None, 6272)	0
dense_6 (Dense)	(None, 512)	3211776
dense_7 (Dense)	(None, 1)	513
Total params: 3,453,121		
Trainable params: 3,453,121		
Non-trainable params: 0		

```
In [16]: from keras import optimizers
```

```
model.compile(loss='binary_crossentropy',
              optimizer=optimizers.RMSprop(lr=1e-4),
              metrics=['acc'])
```

6.6. Chapter Summary

This chapter provided a comprehensive overview of the technical development and practical implementation of the Food Classification System, bridging the gap between theoretical design and a functional application. The process began with a strategic selection of tools, prioritizing Python as the primary programming language due to its powerful libraries, alongside essential hardware like a high-definition webcam and a capable processor for real-time computations. The core of the development focused on the Coding and Model Building phase, where a specialized 4-layer Convolutional Neural Network (CNN) was scripted to extract and learn features from food images. Furthermore, the User Interface (UI) was seamlessly integrated into the live camera feed using OpenCV, allowing the system to display real-time labels and confidence percentages directly on the video frame. By successfully combining complex backend logic with a minimalist and efficient UI, the implementation phase ensured that the system is not only technically sophisticated but also user-friendly and ready for real-world testing.

Chapter 7

TESTING

7.1. Introduction

Testing is a critical phase where I verify if the food classification system works as expected. The primary goal is to ensure that the CNN model accurately distinguishes between a Pizza and a Burger and that the OpenCV interface displays these results correctly in real-time. This chapter explains the methods, strategies, and specific cases used to validate the system.

7.2. Testing Methods

I used two main methods to test the application:

- **Black-Box Testing:** I tested the system's functionality without looking at the internal code. I simply showed different food items to the camera to see if the output label was correct.
- **White-Box Testing:** I examined the internal structure of the CNN, checking the behaviour of individual layers and ensuring that the data flows correctly from the input layer to the final prediction.

7.3. Software Evaluation

The software was evaluated based on its Accuracy and Loss metrics. I used a validation dataset (images the model had never seen during training) to see how well it could generalize. A high accuracy score and a low loss value indicated that the software was ready for real-world use.

7.3.1. Testing Strategy

My strategy involved a step-by-step approach to catch errors early:

- **Requirement analysis:** I confirmed that the system meets the original requirement of real-time classification with at least 90% accuracy.
- **Test Planning:** I created a schedule to test the model under different lighting conditions and angles.

- **Functional testing:** I verified that basic features, like the "Start Camera" button and "Result Overlay," function correctly.
- **Integration testing:** I checked if the Python backend, the trained .h5 model, and the OpenCV video feed work together without crashing.
- **Performance testing:** I measured the "Inference Time" (the time it takes to predict a frame) to ensure the live scan is smooth and does not lag.

7.3.2. Test Plans

The test plan served as a roadmap. It defined what needed to be tested (the model and the UI), who would test it (the developer), and the environment (local PC with a webcam). It also established the "Pass" criteria the system must correctly label the food in under 200 milliseconds.

7.3.3. Test Cases

Below is a summary of the specific test cases conducted:

Table 7. 1 Test case 01

Test Case ID	Description	Expected Result	Status
TC-01	Show a clear image of a Pizza	System displays "Pizza" with high confidence	PASS

Table 7. 2 Test Case 02

Test Case ID	Description	Expected Result	Status
TC-02	Show a clear image of a Burger	System displays "Burger" with high confidence	PASS

Table 7. 3 Test Case 03

Test Case ID	Description	Expected Result	Status
TC-03	Show a non-food item (e.g., a book)	System shows low confidence or no label	PASS

Table 7. 4 Test Case 04

Test Case ID	Description	Expected Result	Status
TC-04	Test in a low-light environment	System should still attempt to identify the shape	PASS

Table 7. 5 Test Case 05

Test Case ID	Description	Expected Result	Status
TC-05	Test with tilted food items	Augmentation layers should help identify correctly	PASS

7.4. Chapter Summary

This chapter detailed the comprehensive testing process used to ensure the reliability and accuracy of the food classification system. By applying various testing methods including functional, integration, and performance testing, I verified that the CNN model and the live camera interface work perfectly together. The successful execution of multiple test cases proved that the system can handle real-world scenarios, such as different angles and lighting conditions, while maintaining high classification accuracy. Ultimately, the testing phase confirmed that the project meets all its technical requirements and is stable enough for final deployment.

REFERENCES

- [1] Python Programming Language Official Documentation. Retrieved from <https://www.python.org/doc/>
- [2] Keras Documentation, Image Data Augmentation and Preprocessing. Retrieved from <https://keras.io/api/preprocessing/image/>
- [3] Kaggle Datasets, Food-101 and Custom Food Image Datasets. Retrieved from <https://www.kaggle.com/datasets>
- [4] TensorFlow, Large-Scale Machine Learning on Heterogeneous Systems. Retrieved from <https://www.tensorflow.org/>